

Lecture 7: Application Security II

Security

Francesco Bariatti

1 April 2026



Universiteit
Leiden
The Netherlands



Table of Contents

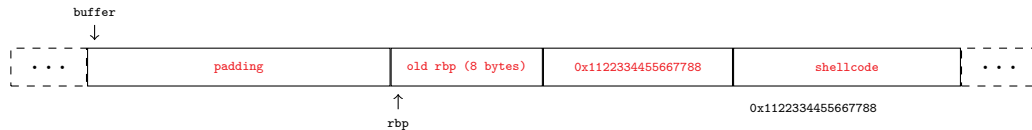
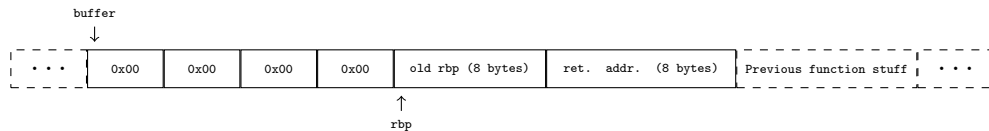
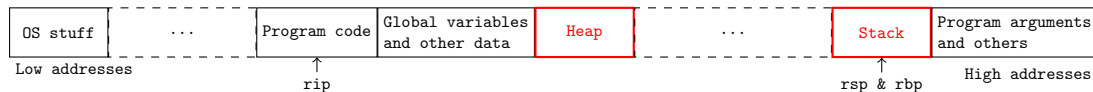
1. Recap
2. Preventing exploits
3. Detecting exploits
 - Control-flow integrity
4. Language-based prevention: Memory safety and Type safety

1. Recap
2. Preventing exploits
3. Detecting exploits
 - Control-flow integrity
4. Language-based prevention: Memory safety and Type safety

Previous lecture recap

- In previous lecture: **attacker perspective**
- How can we exploit a vulnerable program?
- What steps do we take to make a vulnerable program do what we want?

Recap: buffer overflow to inject attacker code



Recap: buffer overflow to affect logic

```

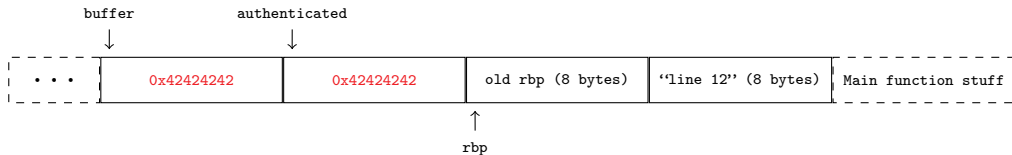
1  int f(char* arg1) {
2      int authenticated = 0;
3      char buffer[4];
4      strcpy(buffer, arg1);
5      if (authenticated) {
6          // Restricted area
7      }
8  }

```

```

9  void main() {
10     char* myStr = "BBBBBBBB";
11     f(myStr);
12     // Rest of the function
13 }

```



Recap: other attacks

- **Heap smashing/overflow**: similar to stack smashing/overflow
 - Different targets, can have different impacts
- **Format string vulnerabilities**
 - Unsanitized user input interpreted as formatting directives
 - Unexpected formatting directives make program behave wrongly
- **Integer over/underflow**: numbers in computers are not infinite!
 - Attackers affects program logic by carefully choosing values
- **Time-Of-Check Time-Of-Use** vulnerability: “race condition” between checking and using property

Attacker steps

As an **attacker**, to exploit a vulnerable program:

- Find some code that we want to execute (but we should not be able to)
 - If not exists, inject our own code as binary instructions
- Access memory beyond theoretical limits (overflow) to modify program status to execute that code
 - If jumping: need to guess code's address
 - Usage of NOP sleds allows for imperfect guesses
- Profit

Goal for this lecture

Defender perspective: how can we make it more difficult for attackers to execute those steps?

Three types of defences:

- **Preventive**: make exploits more **difficult** or impossible
- **Detective**: detect exploits happened and abort execution
- **Reactive/Corrective**: fix state after exploit happened

1. Recap
2. Preventing exploits
3. Detecting exploits
 - Control-flow integrity
4. Language-based prevention: Memory safety and Type safety

To **prevent** exploits:

- Make exploitation **steps** more **difficult or impossible**
- **Avoid bugs** via secure coding and better **code audit**
 - Automated tools, limit to subsets of C, ...
- **Improve architecture**: improve OS, libraries, compiler, ...
 - Make exploits of *all* programs more difficult

⇒ These strategies are **complementary** and are all widely used

To **prevent** exploits:

- Make exploitation **steps** more **difficult or impossible**
- **Avoid bugs** via secure coding and better **code audit**
 - Automated tools, limit to subsets of C, ...
- **Improve architecture**: improve OS, libraries, compiler, ...
 - Make exploits of *all* programs more difficult

⇒ These strategies are **complementary** and are all widely used

Non-executable memory

Attacker steps

Inject our own instructions as binary on the stack. Overwrite `rip` so that execution moves to the stack.

Defences?

Non-executable memory

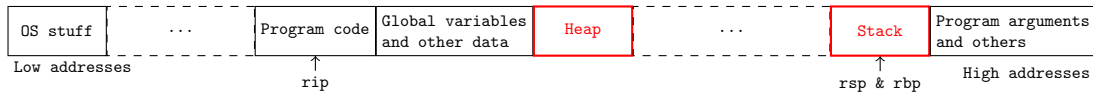
Attacker steps

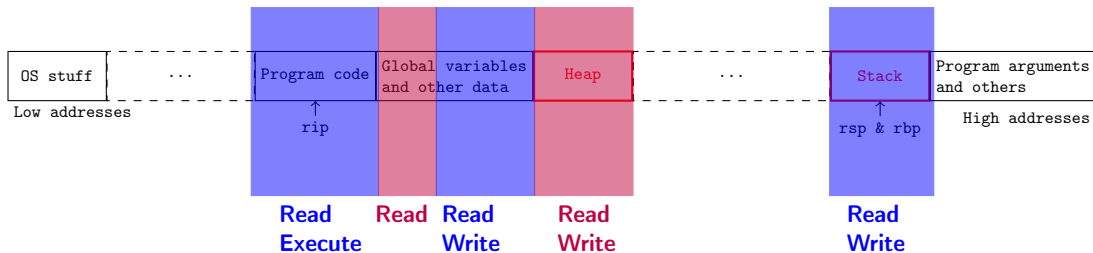
Inject our own instructions as binary on the stack. Overwrite `rip` so that execution moves to the stack.

Defences?

- In modern systems CPU does not access memory directly → Memory Management Unit (MMU)
 - Or Memory Protection Unit (MPU) for smaller systems
- OS can mark stack and heap **segments**¹ as *non-executable*
- CPU refuses to execute instructions from non-executable segments
 - **Segmentation fault**

¹Modern systems use *pages* of memory and not *segments*, but you can think of a segment as a sequence of pages.





Return-oriented programming (ROP)

Does non-executable memory solve everything? What can it *not* solve?

Return-oriented programming (ROP)

Does non-executable memory solve everything? What can it *not* solve?

- Attacker can still overwrite stack content if buffer overflow available
 - In particular, return addresses

Return-oriented programming (ROP)

Does non-executable memory solve everything? What can it *not* solve?

- Attacker can still overwrite stack content if buffer overflow available
 - In particular, return addresses
- Attacker can “frankenstein” their payload from existing instructions in executable memory

Return-oriented programming (ROP)

Does non-executable memory solve everything? What can it *not* solve?

- Attacker can still overwrite stack content if buffer overflow available
 - In particular, return addresses
- Attacker can “frankenstein” their payload from existing instructions in executable memory
- libc (standard C library) particularly juicy target: **return-to-libc attacks**
 - e.g. contains function to execute command (`system`), contains string `"/bin/sh"`
 - Contains all sort of functions with all sorts of instructions: e.g. loading register value from stack
 - **Has well-known structure at well-known addresses**

ASLR

Attacker steps

Exploit the fact that libc is loaded at well-known address and jump to it.

Defences?

ASLR

Attacker steps

Exploit the fact that libc is loaded at well-known address and jump to it.

Defences?

Address Space Layout Randomization (ASLR): place standard libraries and other useful elements randomly in memory

- 👍 Prevents return oriented programming (and return-to-libc in particular)
- 👍 Prevents guessing return address in “standard” stack smashing
 - NOP sled of little help if random space too big

Currently deployed on all OSs

ASLR caveats

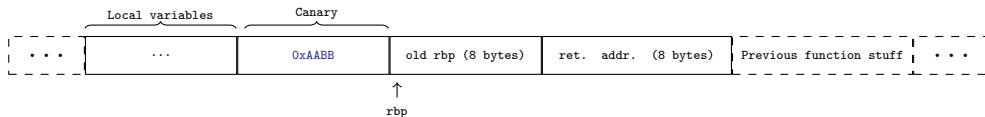
- ☹️ Only randomizes start positions, not contents
- ☹️ To be applied to program instructions, compiler needs to support it
 - 👍 All major compilers and OSs should support it nowadays
- ☹️ Needs enough randomness in the possible addresses
 - 👍 Should not be a problem with 64-bit address space
 - Needs to actually be random
 - ☹️ Attackers can try to reduce possibilities with information leak (e.g. printf format vulnerabilities)
 - ☹️ Memory alignment requirements can reduce possibilities

1. Recap
2. Preventing exploits
3. Detecting exploits
 - Control-flow integrity
4. Language-based prevention: Memory safety and Type safety

Stack canaries

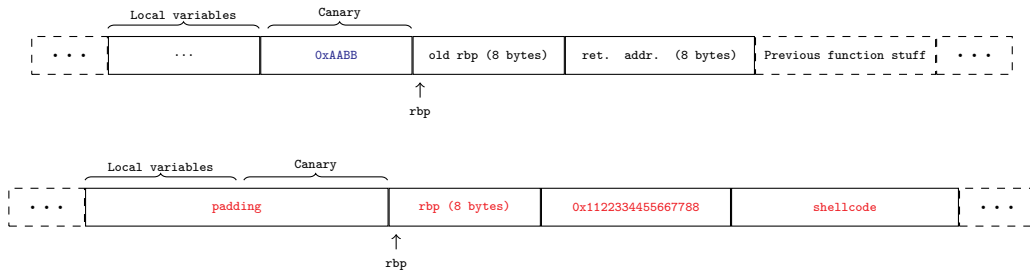
Stack canaries

A **canary** is an alert system that fails early before more important systems fail.



Stack canaries

A **canary** is an alert system that fails early before more important systems fail.



- Compiler adds code to every function to check canary value before loading return address
- If loaded canary does not have expected value: immediately abort

Canary values

What should we set the canary value to?

- Definitely not a well-known value that the attacker can easily inject!

Canary values

What should we set the canary value to?

- Definitely not a well-known value that the attacker can easily inject!
- **Terminator canaries**: contain null or other (e.g. `\n`) bytes that can not be injected via classic vulnerable functions (e.g. `strcpy`, `scanf`)

Canary values

What should we set the canary value to?

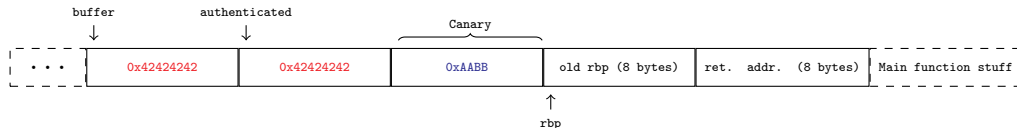
- Definitely not a well-known value that the attacker can easily inject!
- **Terminator canaries**: contain null or other (e.g. `\n`) bytes that can not be injected via classic vulnerable functions (e.g. `strcpy`, `scanf`)
- **Random canaries**: generated at runtime for each process.
 - True value stored somewhere else than stack.
 - Default for gcc when using canaries (default if “sensitive” functions called)

Canary values

What should we set the canary value to?

- Definitely not a well-known value that the attacker can easily inject!
- **Terminator canaries**: contain null or other (e.g. `\n`) bytes that can not be injected via classic vulnerable functions (e.g. `strcpy`, `scanf`)
- **Random canaries**: generated at runtime for each process.
 - True value stored somewhere else than stack.
 - Default for gcc when using canaries (default if “sensitive” functions called)
- **Random XOR canaries**: random value XOR-ed with control flow info (e.g. return address mask)
 - Protects against attacker targeting saved `rip` directly via non-buffer-overflow pointer manipulation

Canary caveats



☹ Does not protect *all* variables against overwriting: just return address (and saved rbp)

- Would need canary after *every* memory location → uses more memory
- Would need code to check after every memory assignment → takes longer

☹ Read overflow could allow attacker to read canary value

- 👍 Increases steps that attacker needs to take and vulnerabilities that need to be present

Behaviour-based detection

- ? How can we **detect** an ongoing attack?
- 💡 If a program **deviates** from its **expected behaviour**, it might be compromised

Behaviour-based detection

- ? How can we **detect** an ongoing attack?
- 💡 If a program **deviates** from its **expected behaviour**, it might be compromised
- ⚠️ What is *expected behaviour*?
- ⚠️ What is *deviation*?
- ⚠️ How to check in a way that is *difficult to compromise*?

Control Flow Integrity (CFI)

- A function consists of **basic blocks** (pieces of code)
- A program execution is made of **control transfers** among basic blocks
 - Here: only look at control transfer between functions
- Connections between basic blocks can be modeled as **Control Flow Graph (CFG)**
- Expected behaviour follows CFG

Call graph

```
void sort(int x[], int len, /*function*/  
↳ f) {  
    // Omitted  
    // Calls f at some point(s)  
}  
  
int sort2(int a[], int b[], int len) {  
    sort(a, len, lt);  
    sort(b, len, gt);  
}
```

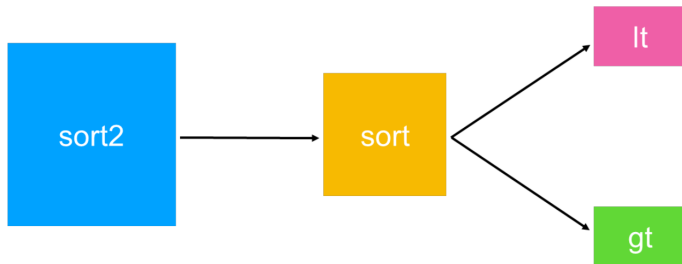
```
bool lt(int x, int y) {  
    return x < y;  
}  
  
bool gt(int x, int y) {  
    return x > y;  
}
```

Call graph

```
void sort(int x[], int len, /*function*/  
→ f) {  
    // Omitted  
    // Calls f at some point(s)  
}  
  
int sort2(int a[], int b[], int len) {  
    sort(a, len, lt);  
    sort(b, len, gt);  
}
```

```
bool lt(int x, int y) {  
    return x < y;  
}
```

```
bool gt(int x, int y) {  
    return x > y;  
}
```



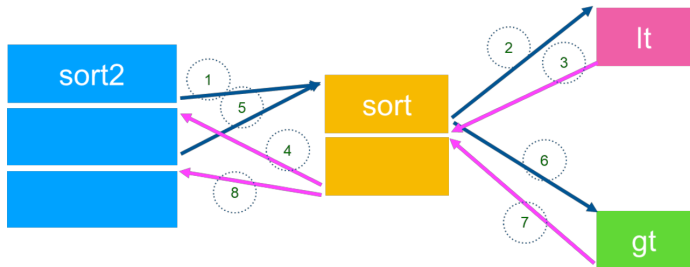
Call graph

```
void sort(int x[], int len, /*function*/
→ f) {
    // Omitted
    // Calls f at some point(s)
}

int sort2(int a[], int b[], int len) {
    sort(a, len, lt);
    sort(b, len, gt);
}
```

```
bool lt(int x, int y) {
    return x < y;
}
```

```
bool gt(int x, int y) {
    return x > y;
}
```



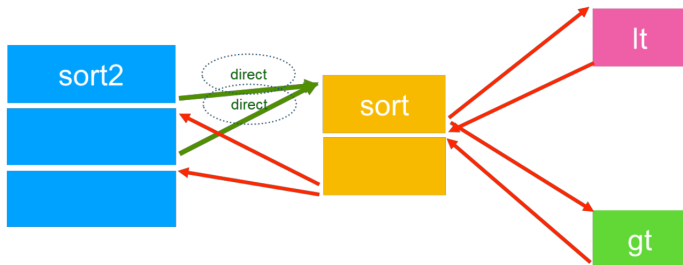
Control Flow Integrity: compliance with CFG

To check whether execution is **compliant**:

- Compute CFG: during **compilation** or by **static analysis** of resulting instructions
 - CFG captures expected behaviour
- **Monitor** behaviour during execution: ensure only paths in CFG are followed
 - **Direct calls** do not need to be monitored (assuming code is immutable)
 - **Indirect calls** (via saved values, based on registers, function tables, etc) need to be monitored
 - Depend on data that can be dynamically altered
 - Monitoring part called **In-line Reference Monitor (IRM)**

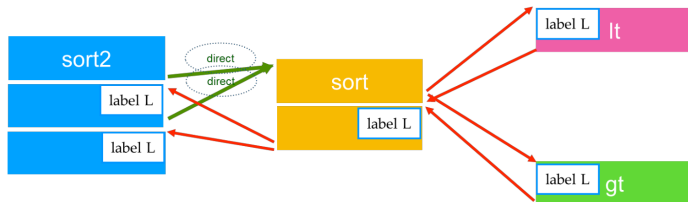
```
void sort(int x[], int len, /*function*/  
→ f) {  
    // Omitted  
    // Calls f at some point(s)  
}  
  
int sort2(int a[], int b[], int len) {  
    sort(a, len, lt);  
    sort(b, len, gt);  
}
```

```
bool lt(int x, int y) {  
    return x < y;  
}  
  
bool gt(int x, int y) {  
    return x > y;  
}
```



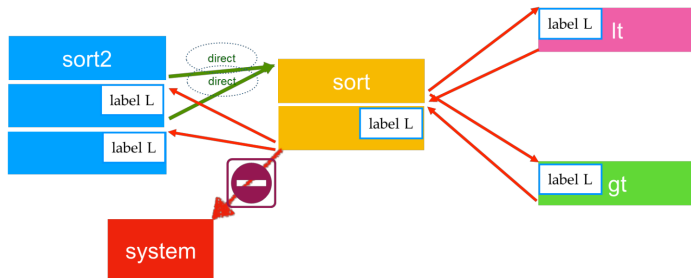
Detecting deviations

- Add instructions to the program to check that current control flow is expected
- Insert a **label** (canary value) before each possible **jump target**
 - In the code, before the first instruction of jump target
- Before jump, check that target address contains label or abort



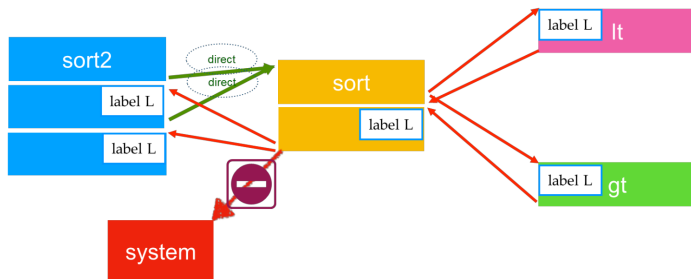
Detecting deviations

- Add instructions to the program to check that current control flow is expected
- Insert a **label** (canary value) before each possible **jump target**
 - In the code, before the first instruction of jump target
- Before jump, check that target address contains label or abort



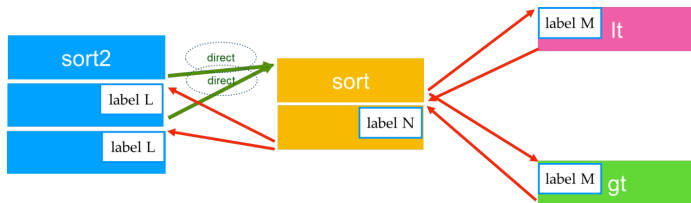
Detecting deviations

- Add instructions to the program to check that current control flow is expected
- Insert a **label** (canary value) before each possible **jump target**
 - In the code, before the first instruction of jump target
- Before jump, check that target address contains label or abort



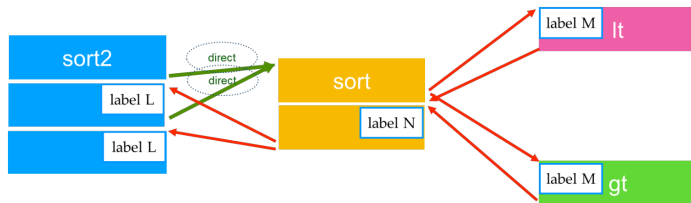
Does not prevent reuse: can jump to any target that is valid *at least once*

Detailed labelling



Constraints on labels:

Detailed labelling



Constraints on labels:

- **lt** and **gt** both called from **sort**: must share a label
- Two possible return sites from **sort**: must share a label

⇒ This is starting to get difficult...

CFI implementation

instrumentation of a destination for jump: add label

Bytes (opcodes)	x86 assembly code	Comment
8B 44 24 04	mov eax, [esp+4]	; first instruction
...		; of destination code
can be instrumented as (a):		
78 56 34 12	DD 12345678h	; label ID, as data
8B 44 24 04	mov eax, [esp+4]	; destination instruction
...		

instrumentation of jump: check label

Bytes (opcodes)	x86 assembly code	Comment
FF E1	jmp ecx	; a computed jump instruction
can be instrumented as (a):		
81 39 78 56 34 12	cmp [ecx], 12345678h	; compare data at destination
75 13	jne error_label	; if not ID value, then fail
8D 49 04	lea ecx, [ecx+4]	; skip ID data at destination
FF E1	jmp ecx	; jump to destination code

Martín Abadi et al. "Control-flow integrity principles, implementations, and applications". In: *ACM Trans. Inf. Syst. Secur.* 13.1 (Nov. 2009), 4:1–4:40. DOI: 10.1145/1609956.1609960

Possible attacks on CFI

- ☹️ Attacker injects code that has a legal label
 - 👍 Not possible for non-executable memory and non-writable code segments
- ☹️ Modify code labels to allow new control flows
 - 👍 Not possible for non-writable code segment
- ☹️ Make it look like a CFI check passed
 - 👍 CFI computations done in registers only: unavailable for attacker
 - 👍 No stack usage for CFI labels

CFI caveats

- 👍 CFI defeats all attacks that change expected control flow: ROP, code injection, ...
- 😞 CFI analysis and instrumentation can be heavy
 - Control flow analysis: very close to halting problem...
 - Check on “hot paths” (functions called many times) can eventually slow down program
- 😞 Still allows control transfer that are compliant
 - **mimicry attacks**
 - Constraints for complex programs may limit actual label diversity
- 😞 Does not prevent data leak or data corruption
 - In particular: read overflow, buffer overflow to affect control flow inside a basic block

1. Recap
2. Preventing exploits
3. Detecting exploits
 - Control-flow integrity
4. Language-based prevention: Memory safety and Type safety

Memory safety

- A **programming language is memory-safe** if all possible programs written in the language are memory-safe
- A **program is memory-safe** if all possible executions are memory-safe

Memory safety

- A **programming language is memory-safe** if all possible programs written in the language are memory-safe
- A **program is memory-safe** if all possible executions are memory-safe

Memory-safe execution:

- **Temporal safety**: only access memory that is **defined** (active)
 - Memory is **undefined** if not yet allocated, uninitialized, deallocated (freed)
- ⇒ Only create pointers through standard means: `p = malloc()`, `p = &x`, ...

Memory safety

- A **programming language is memory-safe** if all possible programs written in the language are memory-safe
- A **program is memory-safe** if all possible executions are memory-safe

Memory-safe execution:

- **Temporal safety**: only access memory that is **defined** (active)
 - Memory is **undefined** if not yet allocated, uninitialized, deallocated (freed)
 - ⇒ Only create pointers through standard means: `p = malloc()`, `p = &x`, ...
- **Spatial safety**: when using a pointer to access memory, only access memory that **belongs** to that pointer

Memory safety

- A **programming language is memory-safe** if all possible programs written in the language are memory-safe
- A **program is memory-safe** if all possible executions are memory-safe

Memory-safe execution:

- **Temporal safety**: only access memory that is **defined** (active)
 - Memory is **undefined** if not yet allocated, uninitialized, deallocated (freed)
 - ⇒ Only create pointers through standard means: `p = malloc()`, `p = &x`, ...
- **Spatial safety**: when using a pointer to access memory, only access memory that **belongs** to that pointer

⇒ Spatial safety ensure that access is to a legal region, temporal safety ensures that content is still valid

Spatial safety

- “Extended” pointer representation (p, b, e)
 - p is actual pointer
 - b is **base** (i.e. start) of memory region that p can access
 - e is **extent** (i.e. end/size) of memory region that p can access

Spatial safety

- “Extended” pointer representation (p, b, e)
 - p is actual pointer
 - b is **base** (i.e. start) of memory region that p can access
 - e is **extent** (i.e. end/size) of memory region that p can access
- Pointer arithmetic can change p , but does not change b and e
- Access allowed if and only if $b \leq p \leq e - \text{sizeof}(\text{typeof}(p))$

Spatial safety

- “Extended” pointer representation (p, b, e)
 - p is actual pointer
 - b is **base** (i.e. start) of memory region that p can access
 - e is **extent** (i.e. end/size) of memory region that p can access
- Pointer arithmetic can change p , but does not change b and e
- Access allowed if and only if $b \leq p \leq e - \text{sizeof}(\text{typeof}(p))$

```
int x;
int* y = &x; //  $p = \text{addr}(x)$ ,  $b = \text{addr}(x)$ ,  $e = \text{addr}(x)+4$  (assuming 4-byte int)
```


Spatial safety

- “Extended” pointer representation (p, b, e)
 - p is actual pointer
 - b is **base** (i.e. start) of memory region that p can access
 - e is **extent** (i.e. end/size) of memory region that p can access
- Pointer arithmetic can change p , but does not change b and e
- Access allowed if and only if $b \leq p \leq e - \text{sizeof}(\text{typeof}(p))$

```
int x;
int* y = &x; // p = addr(x), b = addr(x), e = addr(x)+4 (assuming 4-byte int)
int* z = y+1; // p = addr(x)+4, b = addr(x), e = addr(x)+4
```

Spatial safety

- “Extended” pointer representation (p, b, e)
 - p is actual pointer
 - b is **base** (i.e. start) of memory region that p can access
 - e is **extent** (i.e. end/size) of memory region that p can access
- Pointer arithmetic can change p , but does not change b and e
- Access allowed if and only if $b \leq p \leq e - \text{sizeof}(\text{typeof}(p))$

```
int x;
int* y = &x; // p = addr(x), b = addr(x), e = addr(x)+4 (assuming 4-byte int)
int* z = y+1; // p = addr(x)+4, b = addr(x), e = addr(x)+4

*y = 42; // OK: addr(x) <= addr(x) <= addr(x)+4-4
*z = 42; // ERROR: addr(x) <= addr(x)+4 > addr(x)+4-4
```

```
char buf[4]; // p = addr(buf), b = addr(buf), e = addr(buf)+4
```

```
char buf[4]; // p = addr(buf), b = addr(buf), e = addr(buf)+4  
y[2] = "A"; // OK: p = addr(buf)+2, addr(buf) <= addr(buf)+2 <= addr(buf)+4-1
```

```
char buf[4]; //  $p = \text{addr}(\text{buf}), b = \text{addr}(\text{buf}), e = \text{addr}(\text{buf})+4$   
y[2] = "A"; // OK:  $p = \text{addr}(\text{buf})+2, \text{addr}(\text{buf}) \leq \text{addr}(\text{buf})+2 \leq \text{addr}(\text{buf})+4-1$   
y[4] = "B"; // ERROR:  $p = \text{addr}(\text{buf})+4, \text{addr}(\text{buf}) \leq \text{addr}(\text{buf})+4 > \text{addr}(\text{buf})+4-1$ 
```

Memory safety violations

- Buffer overflow is spatial safety violation: accessing memory beyond e
- Format string vulnerabilities can allow spatial safety violations

Memory safety violations

- Buffer overflow is spatial safety violation: accessing memory beyond e
- Format string vulnerabilities can allow spatial safety violations
- Use-after-free is a temporal safety violation
- Holding a pointer pointing inside a function's stack after the function returns is a temporal safety violation

Memory safety violations

- Buffer overflow is spatial safety violation: accessing memory beyond e
- Format string vulnerabilities can allow spatial safety violations
- Use-after-free is a temporal safety violation
- Holding a pointer pointing inside a function's stack after the function returns is a temporal safety violation
- Integer overflows do not violate memory safety *as long as they don't cause an illegal pointer to be made*

```
1 unsigned short x = 65535;  
2 x = x+1; // Overflows and becomes 0  
3 printf("%d\n",x); // OK: memory-safe  
4 char* p = malloc(x); // Allocates size-0 buffer  
5 p[65535] = "A"; // ERROR
```


Memory-safe languages

- Many languages will take care of memory safety for you
 - Python, Java, C#, Rust, ...
 - Rust explicitly implements “borrow checker”: checks temporal safety at compile-time by automatically marking pointers as invalid
- ⇒ Easiest way to avoid buffer overflow vulnerabilities
- Most of the time: add checks at runtime
- C and C++ are **not memory-safe**
- **C and C++ are not going away**
 - Although rustacean are trying really hard to make it so :)
- Possible to write memory-safe C/C++: compiler-added checks for memory violations
 - Not universally used due to performance penalties
 - Many ways to create a wrong pointer in C

Type safety

- Each object has a type: int, pointer to int, struct, pointer to function, etc
 - Implicit in some languages, but definitely there
- **Type safety** ensures that operations on objects are always compatible with types
 - Injecting shellcode in a buffer and jumping via a valid buffer pointer is *technically* memory-safe w.r.t. the pointer. . . but not type-safe!
- Dynamically-typed languages (e.g. Python) have additional metadata to identify the type of each object
 - If an operation is not possible for the **current** type, throw exception
 - Well-defined behaviour and thus type-safe

Costs of safety

Type and memory safety enforcement is expensive computationally

- Garbage-collected languages avoid temporal violations, but require more memory (and time) than malloc/free
- Runtime bounds check take time on each access but avoid spatial violations
- Verifying violations at compile time removes runtime costs but increases compilation costs
 - Generally a good tradeoff → Rust

- [Aba+09] [Martín Abadi et al.](#) “Control-flow integrity principles, implementations, and applications”. In: *ACM Trans. Inf. Syst. Secur.* 13.1 (Nov. 2009), 4:1–4:40. DOI: 10.1145/1609956.1609960.